Project: UPL to Isabelle/HOL compiler

March 9, 2026

Abstract

The goal of the project was to make a first in-road implementing a compiler from the UniFormal Programming Language (UPL) to Isabelle/HOL. UPL is programming language that instantiates the UniFormal language framework with functional, imperative and object-oriented programming features for modelling mathematical theories in a uniformal, foundation-independent way. Isabelle is a proof assistant with a functional programming language.

UPL is still at the experimental development stage and represents an evolution of MMT. Its distinctive features are open and closed theories and the separation of identifiers into global, regional and local contexts. The theoretical challenge for a compiler is the translation of UniFormal's abstract syntax to Isabelle/HOL's abstract syntax. In practice, the implemented compiler translates UPL's abstract syntax tree (AST) to an intermediate representation of Isabelle/HOL that is unparsed to Isabelle's concrete syntax.

Related to the abstract syntaxes are the language constructs of both languages and their correspondences and degree of equivalence. The compiler covers the UPL AST and the AST coverage determines the compiled language fragment. The compiler covers only FP features of UPL and leaves the OOP features not yet implemented. The report contains a table that summarizes the implementation status of all the AST nodes.

The compiler's architecture follows a structured pipeline. It begins with a command-line interface (CLI) and file I/O, proceeds through UPL parsing and checking, AST-to-IR compilation, and concludes with IR-to-string transformation. The two main parts of the implementation are the design of the intermediate representation (IR) and the compilation from UPL AST to IR. The IR is designed for pragmatic code generation and is very close to the UPL AST with no direct relation to Isabelle/HOL's internal representation. The compiler more or less follows the structure of the UPL AST with a top-level method for compiling a single UPL module that recursively calls the methods for compiling declarations, expressions, types, operators and names. Moreover, the implementation includes UPL modules, numbers and basic arithmetic, collections, a feature traverser for package indexing, functions, Isabelle locales, proof types and terms and reserved words.

The report also summarizes the changes to the UPL code base and the testing phase. The last section discusses directions for future work. These include the CLI, subtyping, recursion, type classes, OOP, and monads.

1 Introduction/Preliminaries

1.1 UPL

The UniFormal Programming Language (UPL) is an experimental programming language that is related to the MMT framework for knowledge representation. OpenMath and MathML are universal representation formats for mathematical knowledge. OMDoc is an extension of them to mathematical theories and documents. MMT is a specification and implementation of the semantics of OMDoc and abbreviates multiple things depending on the current focus or emphasis: Module System for Mathematical Theories, modular mathematical theories, meta-meta-theory and meta-meta-tool. UniFormal is a simple framework language for representing formal theories and models. It improves the handling of identifiers in MMT by systematically accommodating both *open* and *closed* theories as well as separating identifiers and associated contexts into three levels: *global*, *regional*, *local*. UPL is an instance of the UniFormal base language with a concrete type and proof system and operational semantics [9].

UPL like MMT is written in the functional and object-oriented language Scala [5]. The MMT language is based on a small set of carefully chosen, orthogonal primitives. The three main language concepts are the theory, constant declaration, and object. A MMT theory is a list of named declarations and uniformly represents all languages including foundations, logical frameworks, logics and type theories, signatures and theories, set theories. MMT constants represent all operators and symbols of a language including logical symbols, types, ontological symbols, and those of calculi. Objects in MMT represent all composed expressions including formulas, derivations, proofs, terms, types, kinds, and universes [8].

UPL is an object-oriented and functional programming language similar to Scala. UPLs modules and theories are similar to modules and classes in object-oriented programming (OOP) languages as a mechanism for structuring programs through abstraction, encapsulation, modularisation and inheritance. Abstract theories can be made concrete analogous to objects that instantiate classes, although UPLs primary use-case is formalizing mathematical knowledge with theories. Thus, the intended use-case for theories is not OOP but encapsulating formal knowledge. Other OOP features include mutable assignments, block expressions, and loops. But UPL also has functional programming (FP) features like lambda and pattern-matching expressions.

There is a paper describing type classes in UniFormal via primitive theories [1].

Figure 1 shows the grammer of UniFormal. UniFormal is a minimal formal language for mathematical knowledge representation and a framework for representing formal theories and models. The grammer describes a simple abstract syntax with theories, contexts and declarations. The three kinds of declarations are theory declarations, symbol declarations and includes. All but the includes are named. Formal systems typically have multiple kinds of named declarations and corresponding kinds of expressions. Typical kinds of expressions are type symbols and types, function symbols and terms, predicate symbols and formulas, or axioms and proofs. [9].

Declarations	
$D \ ::= [\textbf{open}]\ \textbf{theory}\ \tau = T$	theory (open or closed)
$\mid\ [\textbf{total}]\ \textbf{include}\ T = [e]$	include
$\mid\ \textbf{type}\ a = [A]$	type symbol, with definiens
$\mid\ \textbf{term}\ c : A = [t]$	term symbol, with type, definiens
Theory Expressions	
$T \ ::= \tau \mid \tau.\ldots.\tau.\tau$	regional/global identifier
$\mid\ e.T$	pushout of T along model e
$\mid\ \{D^*\}$	anonymous theory
Types	
$A \ ::= x \mid a \mid \tau.\ldots.\tau.a$	local/regional/global identifier
$\mid\ e.A$	interpretation of A in model e
$\mid\ Mod(T)$	models of T
$\mid\ \blacktriangleright\ t$	proofs of $t : \texttt{Bool}$
$\mid\ \texttt{Bool} \mid \texttt{Int} \mid A^* \to A \mid \ldots$	example base types
Terms	
$e, t ::= x \mid c \mid \tau.\ldots.\tau.c$	local/regional/global identifier
$\mid\ e.t$	interpretation of t in model e
$\mid\ mod(T)$	a concrete model
$\mid\ \uparrow^n$	model of n-th enclosing region (for $n \in \mathbb{N}$)
$\mid\ \ldots$	other productions as needed
Contexts	
$\Gamma \ ::= D\ ([\mid]\ T\ L)^*$	global+stack of (regional+local)
$L \ ::= D^*$	local context of bound variables

Figure 1: UniFormal Syntax

The design and implementation of UPL is a specialization and instance of UniFormal to a programming language. It injects suitable type and proof systems and operational semantics. Next to theories and theory expressions, UPL implements, type and term declarations where axioms and theorems are special cases of terms [9].

1.2 Isabelle

Isabelle is a (semi-)automated theorem prover with applications in computer-assisted theorem proving, formalizing mathematical knowledge, and formal verification of functional programs [6]. Actually, it is inaccurate to say that Isabelle is a proof assistant. More specifically, Isabelle is a foundation-independent framework and compiler for specifying logical foundations and implementing theorem provers. Isabelle/Pure refers to the logical framework for building logics and Isabelle/HOL is an object logic of the Isabelle framework that implements higher-order logic (HOL). Isabelle/HOL's type system is similar to FP languages like Haskell and provides mathematical types and proof tools for theorem proving. It has a vast library of existing theorems and developments in the Archive of Formal Proofs [3, 4].

[7] is a previous effort to represent Isabelle in LF. The paper solves the logical questions of the representation although the software engineering questions of the translation remain non-trivial and open. It even accomplishes to represent advanced Isabelle features like type classes and locales by extending LF conservatively with a module system that adds morphism variables and abstraction over them.

Figure 2 shows a simplified grammer for Isabelle from this effort.

theory	::=	`theory` *name* `imports` *name** `begin` *thycont* `end`
thycont	::=	(*locale* | *sublocale* | *interpretation* |
	|	*class* | *instantiation* | *thysymbol*)*
locale	::=	`locale` *name* = (*name* : *instance*)* for *locsymbol** + *locsymbol**
sublocale	::=	`sublocale` *name* < *instance proof**
interpretation	::=	`interpretation` *instance proof**
instance	::=	*name* `where` *namedinst**
class	::=	`class` *name* = *name** + *locsymbol**
instantiation	::=	`instantiation` *type* :: (*name**)*name* `begin` *locsymbol** *proof** `end`
thysymbol	::=	`consts` *con* | `defs` *def* | `axioms` *ax* | `lemma` *lem*
	|	`typedecl` *typedecl* | `types` *types*
locysymbol	::=	`fixes` *con* | `defines` *def* | `assumes` *ax* | `lemma` *lem*
con	::=	*name* :: *type*
def	::=	*name* : *name var** $\equiv$ *term*
ax	::=	*name* : *Prop*
lem	::=	*name* : *Prop proof*
typedecl	::=	(*var**) *name*
types	::=	(*var**) *name* = *type*
namedinst	::=	`name` = *term*
type	::=	*var* :: *name* | *name* | (*type*, ..., *type*) *name* | *type* $\Rightarrow$ *type* | *prop*
term	::=	*var* | *name* | *name term** | λ(*var* :: *type*)*.*term*
Prop	::=	*Prop* $\Longrightarrow$ *Prop* | $\bigwedge$(*var* :: *type*)*.*Prop* | *term* $\equiv$ *term*
proof	::=	a primitive Pure inference as defined in [21, p. 7]
name, *var*	::=	identifier

Figure 2: Simplified Isabelle Grammer

1.3 Motivation

At this stage in the development, UPL has no implementation of proof terms beyond the placeholder term `???`. UPL also has no proof-checking or automated theorem proving. A compilation to Isabelle/HOL would allow proving with Isabelle and open the possibility of implementing proof terms as Isabelle proofs.

Furthermore, properly implementing compilers to another language systems with the associated software engineering and testing develops the language by exposing incompleteness and bugs as well as supplying other ideas for extending the language. For example, with Isabelle-like typeclasses. Implementing a compiler also has a teaching purpose and learning effect of understanding the UPL and Isabelle system, their syntaxes and translation.

For this project, implementing a compiler from UPL to Isabelle meant translating UPL code to Isabelle and Isabelle/HOL code. It is primarily concerned with questions of syntax and its representations. More specifically, the concrete syntaxes of the two languages and their abstract syntaxes in the internal and intermediate representations. The concrete syntaxes and the abstract syntax of the source language are already given and temporarily fixed. Thus, the main task of the compiler is to design an intermediate representation (IR), translate the abstract syntax tree (AST) of UPL into the IR and unparse the IR into Isabelle's concrete syntax.

Previous work directly related to this task is prototype code for a compiler to JavaScript in the UPL code base consisting of the two files `Javascript.scala` and `Compiler.scala`. The first file implements the IR and its unparsing to Javascript's concrete syntax. The other file implements the translation of UPL's internal representation (AST) to the Javascript IR. The UPL parser parses UPL's concrete syntax into its AST.

Similarly, the project contributed two files `IsabelleSyntax.scala` for the IR of Isabelle's surface-level syntax and `IsabelleCompiler.scala` for the compilation from UPL's AST to the IR. Additionally, there are contributions to `Main.scala`, `Project.scala` and `Traverser.scala` that are concerned with calling the compiler from the command-line and traversing the internal representation to index the packages for the imports statement of the Isabelle theory.

2 UPL to Isabelle/HOL compiler

Generally, the goal was to implement a compiler from UPL to Isabelle/HOL that translates UPL code into Isabelle/HOL code that correctly type-checks. Thus, the aim was to create a first prototype of such a compiler that represents a first-in road with a few working examples, e.g. the AI search algorithm.

The result is Scala 2 code that contributes to the UPL code base and compiles UPL project files (`.p` extension) to Isabelle/HOL theory files (`.thy`). The implemented compiler translates the UPL AST into a surface-level Isabelle/HOL IR, which is distinct from the internal representation of the Isabelle implementation and only serves the purpose of immediate and pragmatic code generation.

2.1 Language constructs

Conceptually, the compiler has the task of translating UPLs language constructs into the equivalent Isabelle/HOL language construct. Broady speaking, the UPL AST is classified into declarations, types, expressions, standard programming language objects and operators. More specifically, UPL has modules, abstract and concrete theories, types, terms, project folders, inheritance, dependent types, quotient types, model types, and the type of terms. Isabelle/HOL is somewhat differently structured and has different language constructs with different names. Isabelle/HOL has theories, locales, interpretations, terms, types, definitions, theorems, proofs et cetera.

Table 1 lists all the UPL language constructs and their Isabelle/HOL equivalents, if they exist.

UPL	Isabelle/HOL
open theories (modules)	theories
(abstract) closed theories	locales
concrete closed theories?	interpretations
terms	terms
types	types
project	
inheritance	
dependent types	
quotient types	
model types	
type of terms	locales/inductive types

Table 1: UPL language constructs and their Isabelle/HOL equivalents

2.2 UPL AST & Language Fragment

The UPL language fragment covered by the compiler can be described in terms of the parts of the AST it covers. The AST nodes are classified by declarations, references, types, expressions, standard programming language objects, and operators.

In the declaration category, the compiler covers modules, theories, include statements, type declarations and expression declarations.

For references with polymorphism, the compiler covers expression references, type references, but no theory references.

The types with a direct translation to Isabelle/HOL are the unit type, bool type, string type, any type (translated to type variables), number types, function types, product types, collection types, and proof types. For types without a direct translation to Isabelle/HOL the compiler throws errors. They are class types, expressions over, exception type, interval types. Unknown types could be translated with type variables but they shouldn't happen because of type checking and inference before the translation.

For expressions the compiler implements lambda, function application, tuple, projection, collection values, list elements, quantifiers, equality, booleans, numbers, strings, base operators, and undefined values. The compiler doesn't implement the expressions for this, instance, expression over, eval, assert, and unit value.

Standard programming language objects that are implemented by the compiler are variable declarations and if expressions. The compiler doesn't implement mutable assignments, blocks, match expressions, for and while loops, and return expressions.

The compiler implements all the operators: logical operators, arithmetic number operators, comparison number operators, list operators (except snoc), equality and inequality operators.

Broadly speaking, the fragment covers the functional programming features but not the imperative and object-oriented programming features of UPL. For example, the compiler translates variable declarations but no mutable assignments, blocks, loops and returns. It translates the standard types common to FP and OOP, but not the types specific to OOP like the class type. For expressions, the compiler translates lambda expressions and function application but not the this and instance values.

Only one toplevel module and no nested modules. Theories not yet implemented, includes and reference between theories. Also no references between modules.

Table 2 summarizes the AST coverage of the compiler by showing the implementation status of the AST nodes for the five AST categories.

Table 2: UPL AST Implementation Status

AST Category	Status and Elements
Declarations	**Implemented:** Module, ExprDecl **Pending:** Include, TypeDecl,
References	OpenRef, ClosedRef, VarRef, AppliedRef
OwnedObject	OwnedExpr, OwnedType, OwnedTheory
Types	**Implemented:** UnknownTypeContainer?!, UnknownType?!, UnitType, BoolType, StringType, NumberType, IntervalType, FunType, (SimpleFunType?!), ProdType, CollectionType, ProofType **Pending:** Type, EmptyType, ClassType, AnyStructure, ExprsOver, BaseType, AnyType, ExceptionType
Expressions	**Implemented:** Lambda, Application, Tuple, CollectionValue, Quantifier, Equality, UnitValue, BoolValue, NumberValue, RealValue, RatValue, IntValue, FloatValue, Real, ApproxReal, Rat, StringValue, String, BaseOperator, UndefinedValue **Pending:** This, Instance, ExprOver, Eval, Projection, ListElem, Assert, BaseValue
Constructs	**Implemented:** VarDecl, Block, IfThenElse **Pending:** Assign, Match, MatchCase, For, While, Return
Operators	**Implemented:** Operator, InfixOperator, PrefixOperator, Associativity, NumberOperator, Arithmetic, Connective, And, Or, Not, Implies, Plus, Minus, Times, Divide, Minimum, Maximum, Power, UMinus, Less, LessEq, Greater, GreaterEq, CollectionOperator, Concat, In, Cons, Equal, Inequal **Pending:** PseudoOperator?!, GeneralInfixOperator, PseudoInfixOperator, PseudoPrefixOperator, PseudoPost, PseudoBind, PseudoCircum, Snoc, InferenceCallbacks

2.3 Processing steps

The implemented compiler is composed of the following processing steps: it first read UPL projects files, parses and checks their content with the UPL parser and checker, translates UPLs internal representation encoded by UPLs AST into a surface-level syntax representation for Isabelle/HOL, and lastly unparses and writes the IR to Isabelle theory files. The following describes these steps in more detail.

Command-line interface

The compiler is called with the command-line flag `--isabelle` followed by the command-line argument for the project file or folder of project files.

File reading and writing

File reading and writing reuses the existing UPL code. The compiler reads the project file or folder of project files (`.p`) and writes to Isabelle theory files (`.thy`) in the same location. The theory files are named after the module names and not the filenames of the project files because Isabelle allows only one theory per file and the theory name and filename must match.

UPL parsing and checking

After file reading, the file content is parsed and checked by the UPL parser and checker. This step reuses the UPL code base by calling the respective methods. Translating only parsed but not checked vocabulary creates multiple problems. First, type-checking errors are not detected and erroneous UPL code is translated. Second, variable declarations with the same name that are merged to a single declaration during checking must be translated to a single Isabelle/HOL declaration. Third, unknown types are inferred during checking.

AST to IR

Translating the UPL AST to the surface-level syntax IR means compiling the checked UPL vocabulary to a Isabelle/HOL IR.

IR to String

The IR is transformed into a string representation for correct and type-checking Isabelle code that can be written to file.

2.4 Intermediate Representation & Compilation

The main task of the implementation was to design an IR for Isabelle and compile UPLs internal representation to it. Generally, the IR is located somewhere in between UPL's and Isabelle/HOL's internal representation. Optimally, it would be as close as possible to the Isabelle/HOL AST of the Isabelle implementation. But that is impractical for a first prototype and the implementation uses an IR very close to UPL and focused instead on the transformation to a string representation of Isabelle/HOL code.

UPL AST

The compiler covers the whole UPL AST and throws errors for AST nodes that are not yet implemented. The UPL AST was described above and the implementation status of the individual AST Nodes was summarized in Table 2. As mentioned above, the compiler implements the functional programming language features of UPL because they have a more or less direct translation to Isabelle's functional language. On the other hand, the compiler does not implement the imperative and object-orented programming features of UPL and throws errors when encountered during compilation.

Intermediate representation

The intermediate representation for Isabelle surface-level syntax more or less mirrors the structure of the UPL AST and is not reflective of Isabelle's internal syntax representation. Instead the focus was on how to transform this representation close to the UPL AST into a string representation for Isabelle/HOL code.

Compiler

The compiler has the task of translating the UPL AST into the IR. In theory, the methods for recursive traversal and compilation should directly reflect the tree structure of the UPL AST. The UPL AST consists of declarations, types, expressions, standard programming language objects and operators. Reflecting this structure, the compiler should have a top-level method for compiling declarations, which calls methods for compiling types, expressions, standard programming language objects and operators.

In practice, the top-level method compiles a single top-level module and nothing else. UPL allows multiple nested and sequential module per project file, but Isabelle theory files must have only a single top-level theory. Therefore, the current implementation only allows a single top-level module per

project file and relegates the handling of nested and consecutive modules to future work.

The top-level method for modules calls the method for names, declarations, and the feature traverser that indexes the packages that the theory must import. The method for compiling declarations in turn calls the methods for types, expressions, operators, and names. Since Isabelle and Isabelle/HOL have many reserved words, a separate method for compiling names that handles the reserved names is necessary. The standard programming language objects are included in the method for expressions.

UPL modules

As just mentioned, the current implementation only allows one top-level module per project files and throws errors for nested and sequential modules. UPL modules are translated to Isabelle theories and Isabelle only allows a single, top-level theory per file. Isabelle theories have an imports statement that imports other theories into the namespace of the theory. The compiler needs to figure out which theories to import. This is achieved by a feature traverser of UPL's internal representation that "indexes the Isabelle packages", which is discussed below.

Numbers & Basic arithmetic

UPL has 9 number types. The compiler currently implements 6 of them: natural numbers, integers, rational numbers, complex numbers, floats and ratCF. The standard Isabelle package `Main` doesn't support the types `rat`, `real`, and `complex`, which are supported by the package `Complex_Main`. Therefore, the compiler must detect their usage and change the imported package. The feature traverser that indexes the packages that the theory must import is described below.

UPL only has one division operator, but Isabelle has three different operators for division that have very strict typing rules: div operator (`div`) for integer division, fractional division (`Fract`) for rationals, and field division (`/`) for reals. The implementation inspects the types of the input and output variables and chooses the fractional when the inputs are integers and the output is a rational and chooses field division otherwise. UPL currently has no div operator.

Similarly, UPL only has one exponentiation operator while Isabelle has a standard power for natural and integer exponents and a real power for real exponents. The compiler inspects the type of the exponent and chooses the commensurate power.

Collections

UPL has five collection kinds: list, set, bag, unique list and option. Isabelle's Main package has options, lists and sets. For bags Isabelle must import a package for multisets (`HOL-Library.Multiset`). Isabelle has no obvious implementation of unique lists, but they could be implemented with the distinct property applied to lists or finite sets (`HOL-Library.FSet`).

UPL has the snoc operator, which adds an element at the end of a list. It has no direct translation to Isabelle/HOL and could be translated via append and wrapping the element in a list.

Feature Traverser

The compiler implements a feature traverser that indexes Isabelle/HOL packages by traversing UPL's internal representation. Currently, the traverser only matches over type and indexes the packages the theory must import. The default imports only the `Main` package. For rational, real, and complex numbers the main package must be changed to `Complex_Main`. UPL bags require the package `HOL-Library.Multiset` next to the main package.

Functions

In UPLs internal representation all functions are lambdas. It is natural to translate them to Isabelle `definitions` with lambda expressions in the definiens.

For the special case of top-level pattern matching in the function body, the compiler can translate to the `fun` command and use the idiomatic FP style of function definition by pattern matching the arguments.

Isabelle locales

UPL theories can be translated to Isabelle locales. The compiler hints at an implementation of abstract and concrete theories to Isabelle locales and locale interpretation.

Proof types and terms

UPL axioms and proofs (`|-`) can be inside theories or outside in the top-level module. The compiler treats them differently by having a separate compile method for inside theories (Isabelle locales). Proofs that are outside are translated to command `theorem` with an associated `apply` proof. Inside theories they are translated to the locale commands `fixes` and `assume`.

UPL has proof types and terms but terms have no implementation beyond the placeholder term `???`. Thus, proof terms are generically compiled to `apply(auto)` and must be completed manually in Isabelle by the user.

Reserved words

Reserved names in Isabelle/HOL can come from multiple sources. These include types (`bool`, `nat`, `set`), values (`False`, `True`, `Nil`), inner syntax keywords (`if`, `let`, `case`), commands or outer syntax keywords (`theory`, `definition`, `lemma`) including proof commands (`apply`, `done`, `try`) and Isar commands (`proof`, `have`, `qed`), proof methods (`auto`, `blast`, `sledgehammer`), package names (`Main`, `Complex_Main`), function and constant names from packages (`length`, `id`, `Suc` from standard library `Main`).

In Isabelle, New names can clash with reserved names if they live in the same namespace. Type name clashes only occur in type declarations that introduce new type names. Terms can have type names without clashing because types and terms have different namespaces. Thus the compiler only needs to annotate type names in type declarations. Value names must be annotated in term-level declarations because Isabelle overwrites them otherwise (shadowing).

Outer syntax keywords (commands, proof commands, Isar commands) can be used for term names because they live in a distinct namespace from inner syntax. On the other hand, inner syntax keywords must be annotated because they live in the same namespace as term names.

Proof methods and package names can be overwritten without consequence because of their scope. Proof methods are only expected as arguments to proof and Isar commands. Package names only occur after the `imports` command.

Reserved function and constant names from the standard library `Main` must be annotated because they are term-level. Isabelle overwrites most of them and throws errors for the others. These include list operations, functional programming terms, and terms for math and logic.

Overall, this shows Isabelle/HOL namespaces distinguish between outer and inner syntax. Commands occur outside and term names inside of quotation marks. Types and terms also have different namespaces, which means types can have term names and terms can have type names without clashing. Therefore, the compiler must only annotate new type names if they use reserved type names and the same for term names.

Furthermore, Isabelle throws inner syntax errors for inner syntax keywords while it overwrites value names and most of the reserved names from the standard library.

The implemented compiler annotates the names by appending an apostrophe (`'`). There are no potential collisions and need for double primes because UPL currently doesn't allow names with apostrophe. Future changes could require changing the annotation method, for example, prepending a letter with underscore (`u_`). A proposal for future work is an additional sanitize step for removing symbols that Isabelle dislikes, for example, dots in qualified names.

2.5 Code changes

In th UPL code base within `package info.kwarc.p` the project modified the files `Main.scala`, `Project.scala`, `Traverser.scala` and created the new files `IsabelleCompiler.scala`, `IsabelleSyntax.scala`.

`Main` extends the command-line interface with the option `--isabelle` for compiling files to Isabelle. `Project` implements the method `toIsabelleFiles` which type-checks the input files, compiles to the IR by calling `IsabelleCompiler` and writes the output files. `Traverser` implements the class and object `IsabellePackageTraverser` that index the packages and create a string for the imports statement.

`IsabelleSyntax` implements the IR and `IsabelleCompiler` the case-matching for the whole UPL AST that compiles the internal to the intermediate representation.

2.6 Tests

The folder `test/isabelle/tests` contains a test suite of small test that are helpful for development and debugging. A single test file can be run with

```
--isabelle test/isabelle/tests/test000.p
```

and the whole folder with

```
--isabelle test/isabelle/tests
```

Table 3 summarizes the content of these test files.

tests	content
test00-09	some initial tests
test20-29	collection type, kinds and values
test30-39	traverser
test40-49	axioms outside of theories
test50-59	operators
test60-69	polymorphism, type variables, any type, type declarations
test70-79	tuples
test80-89	arithmetic operators
test100	locales
test200	checker merges declarations with same name
test300	functions

Table 3: UPL language constructs and their Isabelle/HOL equivalents

Another test suite is within the folder `basics` and can be run with `--isabelle test/isabelle/basics/`. It is taken from the UPL example `basics.p`. Since the compiler only translates part of UPL, there is a file `basics-part.p` that comments out all the parts that currently don't compile.

The folder `targets` contains Isabelle/HOL target code. During development these files were used as an aim for compiling some of the test files.

The folder `negative-tests` contains tests that fail for various reasons. These include nesting of modules, object-oriented programming features, interval types, (mutual) recursion, subtyping, and theories/locales. These failing tests reflect the issues and challenges for translation: No direct correspondence between UPL module and Isabelle theory, No while and for loops, (mutual) recursion with stack overflow, no empty type, no interval types, no snoc, no exceptions, no imperative program blocks and no mutable variable assignment.

3 Issues, Limitations, Future Work

This last section enumerates issues, limitations, and directions for future work regarding the UPL to Isabelle/HOL compiler implementation.

1. **Command-line interface**. Presently, the cli flag `--isabelle` only allows passing a single argument for the input files. Future work could implement an additional argument for output files and another argument for a main expression that is translated to Isabelle/HOL and evaluated in the context of the project by calling Isabelle. Inside Isabelle expressions can be evaluated with the `value` command.

2. **Subtyping**. UPL's subtyping is different to Isabelle/HOL's. UPL collections have subtyping without an equivalent in Isabelle. For example the `in` operator works for all collections in UPL while membership $\in$ in Isabelle/HOL works only for sets. Similarly for number types, UPL has subtyping for number types while Isabelle is more strict. Future work on this front would need to implement the details of Isabelle's type elaborator and coercions.

3. **Recursion**. Currently, UPL functions are translated to Isabelle definitions and lambda expressions. Future work could examine other commands for functions like `fun` and `primrec`. Compiling (mutually) recursive functions requires an `Analyzer` that detects recursion and mutual recursion.

4. **Typeclasses**. UPL currently has no implementation of type classes although they theoretically exist in UniFormal via primitive theories [1]. Isabelle has Haskell-style type classes [2] and future work could implement Isabelle-like type classes for UPL that can then be compiled to idiomatic Isabelle.

5. **OO**Π. There are some Isabelle packages for simulating imperative and OOP language features. Future work could try to also compile those features. These include mutable assignment and variables, while and for loops, blocks, objects, classes, inheritance and qualified names. The `AI.p` example implements imperative algorithms and the work could aim at a successful compilation of those.

6. **Isabelle/HOL monads**. Future work could aim at implementing monads in UPL and compiling them to Isabelle/HOL.

References

[1] Anonymous. Type classes via primitive theories (short paper). *Leibniz International Proceedings in Informatics*, 2012.

[2] Florian Haftmann. Haskell-style type classes with isabelle/isar, 2013.

[3] Tobias Nipkow. Programming and proving in isabelle/hol. In *Technical report, University of Cambridge*. 2013.

[4] Tobias Nipkow, Markus Wenzel, and Lawrence C Paulson. *Isabelle/HOL: a proof assistant for higher-order logic*. Springer, 2002.

[5] Martin Odersky, Lex Spoon, and Bill Venners. *Programming in scala*. Artima Inc, 2008.

[6] Lawrence C Paulson. *Isabelle: A generic theorem prover*. Springer, 1994.

[7] Florian Rabe. Representing isabelle in lf. *arXiv preprint arXiv:1009.2794*, 2010.

[8] Florian Rabe. Mmt: A foundation-independent logical framework. *Online Documentation*, 2018.

[9] Florian Rabe. Global, regional, and local contexts. In *International Conference on Intelligent Computer Mathematics*, pages 240–257. Springer, 2025.